SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

I _____HARI CHARAN P(192511107)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
HARI CHARAN P(192511107)

Annexure A
Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)
A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

---

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation
A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

---

3. Debugging and Optimizing Digital Signature Algorithm (DSA)
A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

---

4. Debugging Key Management System
A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.
Analyze how poor key management can compromise an entire cryptographic system.

---

5. Optimization of Secure File Transfer Protocol (SFTP-like System)
A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.
Fix errors in packet handling and integrity verification.
Optimize throughput for large file transfers (≥100 MB).
Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

		and documentation			

Answer to Question 1: Debugging Hash Function Implementation (SHA Family)

1. Problem Statement

A Python implementation of SHA-256 returns different or malformed digests for the same input. The aim is to (a) find the faults in message padding and block processing, (b) fix the bitwise and endianness errors, (c) speed up hashing of files of 50 MB or more, and (d) explain why a secure hash protects integrity better than a basic checksum. Every result below was produced by running the code and comparing it against the reference implementation in Python's hashlib module.

2. Debugging: Errors Identified and Root Causes

The faulty program was tested on inputs of length 0, 3, 55, 56, 64 and 1100 bytes. These lengths were chosen because 55, 56 and 64 are the boundaries where padding spills into an extra block. The buggy version failed on every input, and its digest for "abc" was thousands of hex characters long instead of 64. Tracing the failures gave six root causes.

#	Location	Defect	Effect	Fix
1	Padding	Length appended with struct.pack("<Q") (little-endian)	Wrong final block, so every digest is wrong	Use ">Q" (big-endian, 64-bit bit-length)
2	Padding	Zero-fill length computed without modulo 64 / off by one at 55–56 byte boundary	Message length not a multiple of 64 bytes	0x80 + zeros * ((55 - len) % 64) + length
3	Block parse	Words read with "<16I" (little-endian)	Message words byte-swapped	Read with ">16I" (SHA is big-endian)
4	Schedule / rounds	No 32-bit mask after additions and rotations	Python ints grow without limit, so rotations are wrong	Apply & 0xFFFFFFFF after every add and rotate
5	Ch function	(~e & g) used with an unbounded ~e	~e is negative in Python, so the bits are wrong	Use (~e & 0xFFFFFFFF) & g
6	Output	'%x' formatting without zero padding or truncation	Variable-length, unmasked digest	Format each word '%08x' after masking

Debugging method. I (i) compared against hashlib on boundary-length inputs, (ii) printed the padded length to confirm it was a multiple of 64, (iii) checked the first message word of the "abc" block (it must be 0x61626380 when read big-endian), and (iv) asserted that every intermediate value stays below 2^32. Each assertion that failed pointed to one of the defects above.

3. Fixed Implementation (Correct Padding, Block Processing and Bitwise Operations)

```
import struct, hashlib
M = 0xFFFFFFFF                                    # 32-bit mask

def rotr(x, n):                                   # masked right-rotate
    return ((x >> n) | (x << (32 - n))) & M

def pad(msg_len):                                 # FIX 1, 2
    return b'\x80' + b'\x00' * ((55 - msg_len) % 64) \
           + struct.pack('>Q', msg_len * 8)       # big-endian bit length

def compress(h, block):
```

```
    w = list(struct.unpack('>16I', block))       # FIX 3: big-endian words
    for t in range(16, 64):
        x, y = w[t-15], w[t-2]
        s0 = rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)
        s1 = rotr(y, 17) ^ rotr(y, 19) ^ (y >> 10)
        w.append((w[t-16] + s0 + w[t-7] + s1) & M)        # FIX 4
    a, b, c, d, e, f, g, hh = h
    for t in range(64):
        S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25)
        ch = (e & f) ^ (~e & M & g)                        # FIX 5
        t1 = (hh + S1 + ch + K[t] + w[t]) & M
        S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22)
        t2 = (S0 + ((a & b) ^ (a & c) ^ (b & c))) & M
        hh, g, f, e, d, c, b, a = g, f, e, (d + t1) & M, c, b, a, (t1 + t2) & M
    return [(x + y) & M for x, y in zip(h, [a, b, c, d, e, f, g, hh])]

def fixed_sha256(data):
    data = bytes(data) + pad(len(data))
    h = H0[:]                                    # initial hash values
    for i in range(0, len(data), 64):
        h = compress(h, data[i:i + 64])
    return ''.join('%08x' % x for x in h)        # FIX 6
```

Verification. After the fixes, the output matched hashlib on all six test inputs (lengths 0, 3, 55, 56, 64 and 1100 bytes). For “abc” the program now prints ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad, the published SHA-256 test vector.

4. Optimisation for Large Files (≥ 50 MB)

The corrected pure-Python version is faithful to the standard but slow, because each 64-byte block needs about 64 schedule steps and 64 rounds of interpreted arithmetic. It hashed 1 MB in 2.47 s (about 0.4 MB/s), so a 50 MB file would take roughly two minutes (extrapolated). Two changes remove this cost:

- **Use the C-backed primitive.** hashlib.sha256 runs the compression function in compiled OpenSSL code (often with CPU SHA extensions). The same 1 MB took 0.00087 s, about 2,800 times faster.
- **Stream the file in fixed chunks.** Reading the whole file into memory costs 50 MB or more of RAM and fails for large files. Streaming with update() keeps memory constant. A reusable buffer and memoryview avoid creating a new bytes object per chunk.

```
def sha256_file(path, chunk=1 << 20):            # 1 MiB chunks
    h = hashlib.sha256()
    buf = bytearray(chunk)
    mv = memoryview(buf)                         # zero-copy slicing
    with open(path, 'rb', buffering=0) as f:
        while (n := f.readinto(buf)):            # no new bytes object per read
            h.update(mv[:n])
    return h.hexdigest()
```

Method	Input	Time	Memory
Fixed pure-Python SHA-256	1 MB	2.47 s (about 124 s for 50 MB, extrapolated)	Whole input in RAM
hashlib, whole-file read	50 MB	0.074 s	About 50 MB
hashlib + 1 MiB streaming (optimised)	50 MB	0.043 s	1 MiB buffer

The streamed digest of the 50 MB random file was identical to the whole-file digest, so the optimisation does not change the result. Streaming is about 1.7 times faster than the whole-file read here and its memory use does not grow with file size. Timings were measured on the evaluation machine and will vary by hardware; the ratios are the meaningful result.

5. Analysis: Secure Hash vs Basic Checksum

A checksum such as a byte-sum or CRC32 is designed to detect accidental errors (noise, a flipped bit). A cryptographic hash such as SHA-256 is designed to resist deliberate tampering. The differences are shown below.

Property	Basic checksum (sum / CRC32)	SHA-256
Output size	8–32 bits	256 bits
Collision resistance	Collisions are trivial to construct (about 2^16 work by birthday attack for 32 bits)	About 2^128 work for a birthday collision; none known
Preimage resistance	Easy: a message can be crafted to match a given value	About 2^256 work; infeasible
Avalanche effect	Weak and linear; CRC is linear over GF(2)	One changed input bit flips about half the output bits
Protects against	Accidental corruption only	Accidental corruption and malicious modification

Worked example. A simple additive checksum gives the same value (131) for “AB” and “BA”, so reordering the bytes of a message is undetected. A CRC32 is only 32 bits long and, being linear, lets an attacker append four chosen bytes to force any target value. SHA-256 gives unrelated 256-bit outputs for these inputs, and no practical method exists to force a chosen digest. Therefore, when a file’s SHA-256 is published through a trusted channel, a matching digest after download gives strong assurance that the file has not been altered, which a checksum cannot give against an active attacker.

Limits. A plain hash proves integrity only if the reference digest itself is trusted. To authenticate the sender as well, the digest should be signed (digital signature) or computed as an HMAC with a shared key. Also, SHA-256 is unsuitable on its own for password storage because it is fast; a slow, salted function such as PBKDF2 or bcrypt should be used there.

6. Conclusion

Six faults (little-endian length and words, missing 32-bit masking, the unbounded ~e, faulty zero-fill and unpadded output formatting) caused the inconsistent digests. After correction the implementation matched the SHA-256 reference on every boundary test. For files of 50 MB and above, streaming through hashlib reduced hashing time from minutes to about 0.04 s with constant memory, with an identical digest. Compared with simple checksums, SHA-256 gives collision and preimage resistance and so protects integrity against deliberate attack as well as accidental error.